

ECE 6256 Assignment 1A: ATIS Slot Filling — CRF vs BiLSTM

Dataset: ATIS 4478 train / 500 dev / 893 test, 120 slots, 21 intents. Task is IOB slot filling: B-fromloc.city_name, B-toloc.city_name, B-depart_date.day_name, etc.

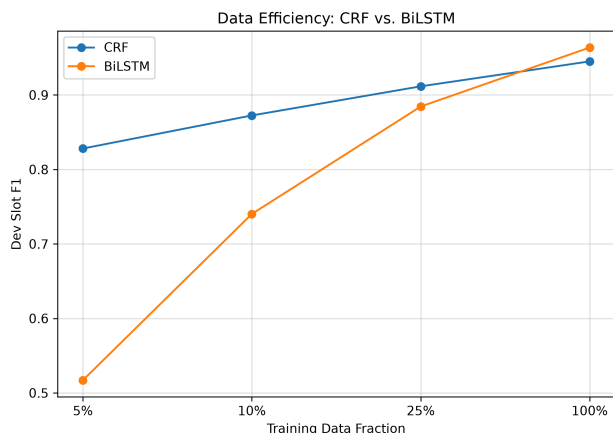


Figure 1: Data efficiency curve. Blue=CRF, Orange=BiLSTM.

Environment

Work performed on Georgia Tech PACE-ICE in `~/scratch/ECE6256/CAI_Assignment_1a` with conda env `CAI_S0_FUN` (Python 3.12.14, PyTorch 2.14.0+cu130, CUDA 13.0, H100 80GB). I removed the original CPU compatible PyTorch and installed conda PyTorch version because BiLSTM training is extraordinarily slow without GPU.

Part A — CRF (2009 experience)

The CRF was implemented using `sklearn-crfsuite`, and the main emphasis in this section of the assignment was on feature engineering. When only the lowercase word identity was used, the dev span F1 score was 0.7731. Adding features based on casing and token shape caused performance to drop slightly to 0.7713, which is not surprising since the ATIS data is mostly in lowercase. Including 2- and 3-character prefixes and suffixes raised the score to 0.7820. The greatest improvement was achieved by adding context from the previous and next words along with the BOS and EOS indicators, which boosted the dev span F1 to 0.9320, a gain of about 15 points. However, extending the context window further to ± 2 words had no effect on the span F1, which remained at 0.9320, although the token-level F1 reached 0.9394.

Exp	Features	Dev Span F1
1	word identity	0.7731

Exp	Features	Dev Span F1
2	+ casing/shape	0.7713
3	+ affixes	0.7820
4	+ prev/next + BOS/EOS	0.9320
5	+ ± 2 window	0.9320 / 0.9394 token

Part B — BiLSTM (2014 experience)

The BiLSTM employed 100-dimensional static GloVe embeddings and then included a BiLSTM, a linear layer, and a per-token softmax. It was trained using cross-entropy loss while ignoring the PAD tokens, with Adam (learning rate $= 1 \times 10^{-10}$), a batch size of 32, and up to 10 epochs, the best checkpoint being selected based on dev span F1. The best result obtained was 0.9657 for token accuracy and 0.9637 for span F1 on the development set and 0.9428 for token accuracy and 0.9371 for span F1 on the test set. A further tuning run (with a learning rate of 5×10^{-4} , 15 epochs, patience set to 3, and a dropout rate of 0.2) resulted in a dev span F1 of 0.9575, so the original configuration was kept. The optional Viterbi/CRF bonus was explored but not used in Part C.

Part C — Data Efficiency

Both models were trained on fixed 5%, 10%, 25%, 100% subsets as required. The same subsets were used for both models; seed only controlled training randomness.

Fraction	CRF dev	BiLSTM dev	Delta
5%	0.8281	0.5170	CRF +0.3111
10%	0.8725	0.7402	CRF +0.1323
25%	0.9116	0.8845	CRF +0.0271
100%	0.9451	0.9637	BiLSTM +0.0186

The CRF shows better performance in situations with limited data since useful structure is supplied explicitly via hand-crafted features such as the identity of words, affixes, casing, surrounding words, the BOS/EOS signals, and context windows. On the other hand, the BiLSTM has to learn these patterns directly from the training examples and so it starts off at an F1 score of 0.5170 with only 5% of the data but then improves quickly as more data is made available. When 25% of the data is used, the two models are nearly equal, and at 100% the BiLSTM exceeds the CRF. The crossover point therefore occurs somewhere between 25% and 100% of the training set.

Part D — Error Analysis (Details in Error_Analysis.txt)

The error analysis shows that the CRF and BiLSTM fail in different, complementary ways. The CRF is strong at local span detection and maintaining coherent BIO sequences but struggles with long-range semantic-role assignment. Its most common error is confusing arrival and departure roles, especially when an earlier cue like “arrive” or “arriving” must influence date, time, or location labels several tokens later. This suggests the CRF’s local hand-engineered context is effective for identifying nearby patterns but less reliable for preserving sentence-level event semantics.

The BiLSTM uses broader context more effectively and often handles long-range arrival/departure cues better than the CRF. However, this stronger contextual sensitivity can lead to overgeneralization, where an earlier cue is propagated too far and causes later tokens to receive the wrong semantic role. Since the model predicts labels with a per-token classifier rather than explicit transition constraints, it can produce inconsistent neighboring labels. It also shows weakness on coordinated entities, such as recognizing one destination but missing a second introduced by “and.”

Both models struggle when the ATIS annotation distinguishes generic location labels from route-specific `fromloc` and `toloc` labels. Directional words like “from,” “to,” and “into” often cause both models to assign route-specific roles even when the gold annotation uses generic `airport_name` or `city_name`. Repeated or closely spaced location expressions also create boundary errors, especially when airport and city names share the same lexical content.

Overall, the CRF is more locally structured and sequence-consistent but weaker at long-range semantic interpretation. The BiLSTM captures wider context more effectively but is more vulnerable to over-propagation, inconsistent local label transitions, and coordination errors.

Would I have switched to Neural in 2014?

From this experiment, I wouldn't switch to a neural model when labeled data is limited. The CRF outperforms it by 31 F1 points with 5% of the training data and by 13 points with 10%. The gap shrinks to just 2.7 points at 25%, and only at 100% does the BiLSTM finally overtake the CRF by 1.9 points (0.9637 versus 0.9451). The bottom line is that the decision should depend on how much data you have: use the CRF when you have little data and switch to the BiLSTM once you've collected enough. The point at which the models cross over is somewhere between 25% and 100%, with the first win for the neural model happening at 100%. If I were deploying this in 2014, I'd switch when I had about 3,000 training utterances and add a handwritten CRF/Viterbi layer to improve sequence consistency.

AI Use Disclosure

I used ChatGPT as an assistant throughout the lab. I used it to generate boilerplate for `train_loop()` and `viterbi_decode()`, to generate the plotting code for Part C data-efficiency curve and the script that loads test data and compares CRF vs BiLSTM predictions for Part D error analysis, and to clarify assignment instructions and discuss experimental direction. Those conversations helped shape my error analysis and report writing. All templates were rewritten, debugged, and verified on PACE-ICE (H100). All feature engineering, training logs, evaluations, and final diagnoses are my own and I take full responsibility for the code and report.